The Queue

Lecture 05

Data Structures and Algorithms

Pranshu Mandal

Part 1: What is a Queue?

The Queue — An Everyday Idea

Think about these scenarios you encounter every day:

- **Movie ticket counter** — the first person in line gets served first
- **Printer queue** — documents print in the order they were sent
- **Customer service calls** — calls are answered in the order they are received
- **Traffic at a toll booth** — cars pass through in the order they arrive

All of these follow the same principle:

The first item added is the first item removed.

This is the **FIFO** principle — **First In, First Out**.

Queue vs Stack: A Quick Contrast

Before diving deeper, let us see how the Queue differs from the Stack we learned last lecture:

```
STACK (LIFO):                          QUEUE (FIFO):
  Only one end                           Two ends

  +------+
  |  30  | <-- top (push/pop here)      Front                Rear
  +------+                             (dequeue)          (enqueue)
  |  20  |                                |                  |
  +------+                                v                  v
  |  10  |                             [10] [20] [30] [40] [  ]
  +------+
```

Stack: One access point (top) — add and remove from the same end

Queue: Two access points — add at the **rear**, remove from the **front**

Queue as an Abstract Data Type

Queue ADT Definition:

Component	Description
Data	A collection of elements with a **front** and a **rear**
Principle	FIFO — First In, First Out

Operations:

Operation	Description
`enqueue(x)`	Add element `x` at the rear of the queue
`dequeue()`	Remove and return the element from the front
`front()` / `peek()`	View the front element without removing it
`isEmpty()`	Check if the queue has no elements
`isFull()`	Check if the queue is full (array-based only)

Queue Operations: Visual Walkthrough

```
Start: Empty queue          enqueue(10)         enqueue(20)         enqueue(30)

  front   rear              front   rear        front      rear     front         rear
   |       |                 |       |           |          |        |              |
   v       v                 v       v           v          v        v              v
 [    ]                     [10 ]               [10 ][20 ]          [10 ][20 ][30 ]
```

```
dequeue() returns 10        dequeue() returns 20        peek() returns 30

    front   rear                front  rear                front  rear
     |       |                   |      |                   |      |
     v       v                   v      v                   v      v
    [20 ][30 ]                  [30 ]                      [30 ]
```

Key observation: Elements leave in the **same order** they arrived — no cutting in line!

Overflow and Underflow

Just like the Stack, Queues have two error conditions:

Queue Overflow:

- Attempting to `enqueue()` when the queue is already full
- Only happens in array-based implementations (fixed size)

Queue Underflow:

- Attempting to `dequeue()` or `peek()` when the queue is empty
- Can happen in any implementation
- Must always check `isEmpty()` before dequeuing

```
OVERFLOW:                                UNDERFLOW:
Queue is full, can't enqueue!            Queue is empty, can't dequeue!

front               rear                 front   rear
 |                   |                    |       |
 v                   v                    v       v
[10][20][30][40][50]                    [  EMPTY  ]
```

Part 2: Queue Implementation Using Arrays

Array-Based Queue: Design

Idea: Use an array of fixed size `MAX`, with two tracking variables:

- `front` — index of the first element (to be dequeued next)
- `rear` — index of the last element (most recently enqueued)

```
Array:  [ _ ][ _ ][ _ ][ _ ][ _ ]     MAX = 5
Index:    0    1    2    3    4
front = 0, rear = -1  (queue is empty)
```

```c
#define MAX 100

struct Queue {
    int arr[MAX];
    int front;
    int rear;
};
```

Initialize: `front = 0`, `rear = -1` (or both to -1, depending on convention).

Array Queue: enqueue() Operation

Algorithm:

1. Check if queue is full (`rear == MAX - 1`) — if yes, overflow error
2. Increment `rear`
3. Place element at `arr[rear]`

```
void enqueue(struct Queue* q, int value) {
    if (q->rear == MAX - 1) {
        printf("Queue Overflow!\n");
        return;
    }
    q->rear++;
    q->arr[q->rear] = value;
}
```

Time Complexity: O(1) — just increment and write.

Array Queue: enqueue() Visualization

```
Initial:    front=0, rear=-1
[ _ ][ _ ][ _ ][ _ ][ _ ]

enqueue(10):   front=0, rear=0
[10 ][ _ ][ _ ][ _ ][ _ ]
  ^
 f,r

enqueue(20):   front=0, rear=1
[10 ][20 ][ _ ][ _ ][ _ ]
  ^    ^
  f    r

enqueue(30):   front=0, rear=2
[10 ][20 ][30 ][ _ ][ _ ]
  ^         ^
  f         r
```

Elements are always added at the rear. The front stays put (for now).

Array Queue: dequeue() Operation

Algorithm:

1. Check if queue is empty (`front > rear`) — if yes, underflow error
2. Read element at `arr[front]`
3. Increment `front`

```
int dequeue(struct Queue* q) {
    if (q->front > q->rear) {
        printf("Queue Underflow!\n");
        return -1;
    }
    int value = q->arr[q->front];
    q->front++;
    return value;
}
```

Time Complexity: O(1) — just read and increment.

Array Queue: dequeue() Visualization

```
Before:     front=0, rear=2
[10 ][20 ][30 ][ _ ][ _ ]
  ^         ^
  f         r

dequeue() returns 10:    front=1, rear=2
[10 ][20 ][30 ][ _ ][ _ ]
       ^    ^
       f    r

dequeue() returns 20:    front=2, rear=2
[10 ][20 ][30 ][ _ ][ _ ]
             ^
            f,r

dequeue() returns 30:    front=3, rear=2  --> front > rear = EMPTY
[10 ][20 ][30 ][ _ ][ _ ]
              ^  ^
              r  f
```

The Wasted Space Problem

Here is the critical problem with linear array queues. Look at what happens:

```
Step 1: enqueue 10, 20, 30, 40, 50    front=0, rear=4 (FULL)
[10 ][20 ][30 ][40 ][50 ]
  ^                   ^
  f                   r

Step 2: dequeue three times            front=3, rear=4
[ _ ][ _ ][ _ ][40 ][50 ]
                 ^    ^
                 f    r

Step 3: Try to enqueue(60)             rear == MAX-1 --> OVERFLOW!
[ _ ][ _ ][ _ ][40 ][50 ]
                 ^    ^
                 f    r
```

The array has 3 empty slots at the front, but we cannot use them!

This is called **false overflow** — the queue reports "full" even though space exists.

Possible Solutions to Wasted Space

Solution 1: Shift Elements Left After Each Dequeue

After dequeuing, shift all remaining elements to the front of the array:

```
Before dequeue:     [10][20][30][40][50]     front=0, rear=4

dequeue() + shift:  [20][30][40][50][ _ ]    front=0, rear=3
```

Problem: Shifting is **O(n)** — every dequeue becomes expensive!
For a queue with 1,000,000 elements, each dequeue moves 999,999 elements.

Solution 2: Circular Queue (next lecture!)

Wrap the array around so that after index `MAX-1`, we go back to index `0`.

This is the elegant solution — all operations stay O(1). We will cover it in detail next lecture.

Array Queue: peek() and isEmpty()

```c
int peek(struct Queue* q) {
    if (q->front > q->rear) {
        printf("Queue is empty!\n");
        return -1;
    }
    return q->arr[q->front];
}

int isEmpty(struct Queue* q) {
    return q->front > q->rear;
}

int isFull(struct Queue* q) {
    return q->rear == MAX - 1;
}

int size(struct Queue* q) {
    if (isEmpty(q)) return 0;
    return q->rear - q->front + 1;
}
```

Array Queue: Complete Example

```c
int main() {
    struct Queue q;
    q.front = 0;
    q.rear = -1;

    enqueue(&q, 10);      // Queue: [10]
    enqueue(&q, 20);      // Queue: [10, 20]
    enqueue(&q, 30);      // Queue: [10, 20, 30]

    printf("%d\n", peek(&q));       // Output: 10 (front element)
    printf("%d\n", dequeue(&q));    // Output: 10, Queue: [20, 30]
    printf("%d\n", dequeue(&q));    // Output: 20, Queue: [30]
    printf("%d\n", isEmpty(&q));    // Output: 0 (not empty)
    printf("%d\n", dequeue(&q));    // Output: 30, Queue: []
    printf("%d\n", isEmpty(&q));    // Output: 1 (empty)
}
```

Notice: Elements come out in the **same order** they went in — 10, 20, 30. Compare this with a stack, where they would come out as 30, 20, 10.

Part 3: Queue Implementation Using Linked List

Why Use a Linked List for Queues?

The array-based queue suffers from:

- Fixed size (must define `MAX`)
- Wasted space problem (false overflow)
- The only fix (shifting) makes dequeue O(n)

A linked list queue solves **all** of these problems naturally:

Problem	Linked List Solution
Fixed size	Grows dynamically — no limit
Wasted space	No gaps — nodes are allocated/freed as needed
Costly shifting	No shifting at all — just pointer updates

Linked List Queue: Design

Key Idea: Maintain two pointers — `front` (head) and `rear` (tail).

- `enqueue()` = insert at the **tail** (rear)
- `dequeue()` = delete from the **head** (front)

Both are O(1) because we track both ends!

```
struct Node {
    int data;
    struct Node* next;
};

struct Queue {
    struct Node* front;
    struct Node* rear;
};
```

```
Queue:  front --> [10|*]--> [20|*]--> [30|NULL] <-- rear
```

Linked List Queue: enqueue() Operation

Algorithm:

1. Create a new node with the given value
2. If queue is empty, set both `front` and `rear` to the new node
3. Otherwise, link the current `rear->next` to the new node, then update `rear`

```
void enqueue(struct Queue* q, int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    if (q->rear == NULL) {
        // Queue was empty
        q->front = q->rear = newNode;
        return;
    }
    q->rear->next = newNode;
    q->rear = newNode;
}
```

Linked List Queue: enqueue() Visualization

```
Initial: front = NULL, rear = NULL  (empty queue)

enqueue(10):
  front --> [10|NULL] <-- rear

enqueue(20):
  front --> [10| *]--> [20|NULL] <-- rear

enqueue(30):
  front --> [10| *]--> [20| *]--> [30|NULL] <-- rear
```

Each enqueue creates a new node and attaches it at the end. The `rear` pointer always advances to the newest node.

Linked List Queue: dequeue() Operation

Algorithm:

1. Check if queue is empty (`front == NULL`) — if yes, underflow error
2. Save the front node's data
3. Move `front` to `front->next`
4. If `front` becomes NULL, set `rear` to NULL as well (queue is now empty)
5. Free the old front node

```
int dequeue(struct Queue* q) {
    if (q->front == NULL) {
        printf("Queue Underflow!\n");
        return -1;
    }
    struct Node* temp = q->front;
    int value = temp->data;
    q->front = q->front->next;

    if (q->front == NULL)
        q->rear = NULL;   // Queue became empty
```

Linked List Queue: dequeue() Visualization

```
Before:
  front --> [10| *]--> [20| *]--> [30|NULL] <-- rear

dequeue() returns 10:
  temp --> [10| *]   (freed)
  front -----------> [20| *]--> [30|NULL] <-- rear

dequeue() returns 20:
  temp --> [20| *]   (freed)
  front -----------> [30|NULL] <-- rear

dequeue() returns 30:     (queue becomes empty)
  temp --> [30|NULL]  (freed)
  front = NULL, rear = NULL
```

Important edge case: When the last element is dequeued, both `front` and `rear` must be set to NULL. Forgetting to update `rear` would leave a dangling pointer!

Linked List Queue: The Single-Element Edge Case

When the queue has exactly one element, `front` and `rear` point to the same node:

```
Single element:
  front --> [42|NULL] <-- rear
            (same node)

dequeue():
  front = front->next = NULL
  rear must also be set to NULL!

Result:
  front = NULL, rear = NULL  (empty queue)
```

This is the most common bug in linked list queue implementations — always check:

```
if (q->front == NULL)
    q->rear = NULL;   // Don't forget this!
```

Linked List Queue: peek() and isEmpty()

```c
int peek(struct Queue* q) {
    if (q->front == NULL) {
        printf("Queue is empty!\n");
        return -1;
    }
    return q->front->data;
}

int isEmpty(struct Queue* q) {
    return q->front == NULL;
}
```

No `isFull()` needed — the linked list queue can grow until system memory runs out.

No wasted space problem — every dequeue frees memory; every enqueue allocates exactly what is needed.

Part 4: Comparing Implementations

Array vs. Linked List Queue

Feature	Array (Linear)	Linked List
Size	Fixed (`MAX`)	Dynamic (grows as needed)
Wasted space	Yes (false overflow problem)	No
Overflow	Possible	Only if system memory runs out
Speed	Slightly faster (cache-friendly)	Slightly slower (pointer chasing)
Code complexity	Simpler, but wasted-space issue	Slightly more complex
Memory per element	Just the data	Data + pointer (extra 4-8 bytes)
Dequeue	O(1) but wastes space, or O(n) with	O(1) always

When to Use Which?

Choose Array-Based Queue when:

- Maximum size is known in advance
- You will use a **circular queue** to avoid wasted space (next lecture)
- Cache performance matters
- Simplicity is preferred

Choose Linked List Queue when:

- Size is unpredictable or highly variable
- You cannot afford to pre-allocate memory
- Wasted space must be avoided without circular logic
- Frequent allocation/deallocation is acceptable

Part 5: Stack vs Queue — The Big Comparison

Stack vs Queue: Side by Side

Feature	Stack	Queue
Principle	LIFO (Last In, First Out)	FIFO (First In, First Out)
Insertion	At top (`push`)	At rear (`enqueue`)
Deletion	From top (`pop`)	From front (`dequeue`)
Access Points	One end (top)	Two ends (front and rear)
Analogy	Stack of plates	Line at a counter
Output order	Reverse of input	Same as input
Primary use	Undo, recursion, DFS, expressions	Scheduling, buffering, BFS

The fundamental difference: a Stack **reverses** the order; a Queue **preserves** it.

Same Input, Different Output

What happens when we insert 1, 2, 3, 4, 5 and then remove all elements?

```
STACK (push 1,2,3,4,5 then pop all):
  Input order:   1, 2, 3, 4, 5
  Output order:  5, 4, 3, 2, 1     <-- REVERSED!

QUEUE (enqueue 1,2,3,4,5 then dequeue all):
  Input order:   1, 2, 3, 4, 5
  Output order:  1, 2, 3, 4, 5     <-- PRESERVED!
```

This reversal property is why stacks are used for:

- Undo operations (reverse the last action)
- Expression conversion (reverse operator order)

And why queues are used for:

- Scheduling (maintain arrival order)
- Buffering (process in order received)

Thought Experiment: Queue Using Two Stacks?

Is it possible to build a Queue using only Stack operations (push/pop)?

Hint: If a stack reverses order, what does **reversing twice** give you?

```
Stack 1 (input stack):     Stack 2 (output stack):

push(1), push(2), push(3)
  [3] <-top                   [empty]
  [2]
  [1]

Transfer: pop all from S1, push to S2:
  [empty]                     [1] <-top    (order restored!)
                              [2]
                              [3]

Now pop from S2: 1, 2, 3    <-- FIFO order!
```

Two reversals cancel out! This is a classic interview question — we will not implement it fully here, but it is worth thinking about.

Part 6: Tracing Through Examples

Trace Exercise 1

What does the queue look like after these operations?

```
enqueue(A), enqueue(B), dequeue(), enqueue(C), enqueue(D), dequeue()
```

Step-by-step trace:

```
enqueue(A):    front=A, rear=A       [A]
enqueue(B):    front=A, rear=B       [A][B]
dequeue():     front=B, rear=B       [B]          returned A
enqueue(C):    front=B, rear=C       [B][C]
enqueue(D):    front=B, rear=D       [B][C][D]
dequeue():     front=C, rear=D       [C][D]       returned B
```

Final Queue: `[C, D]` with `C` at the front and `D` at the rear.

Trace Exercise 2

What is the output of the following code?

```
struct Queue q;
q.front = 0; q.rear = -1;

enqueue(&q, 10);
enqueue(&q, 20);
printf("%d ", dequeue(&q));
enqueue(&q, 30);
enqueue(&q, 40);
printf("%d ", dequeue(&q));
printf("%d ", dequeue(&q));
enqueue(&q, 50);
printf("%d ", dequeue(&q));
printf("%d ", dequeue(&q));
```

Trace Exercise 2: Solution

```
enqueue(10):            [10]                    front=0, rear=0
enqueue(20):            [10][20]                front=0, rear=1
dequeue() -> print 10:  [20]                   front=1, rear=1
enqueue(30):            [20][30]                front=1, rear=2
enqueue(40):            [20][30][40]            front=1, rear=3
dequeue() -> print 20:  [30][40]               front=2, rear=3
dequeue() -> print 30:  [40]                   front=3, rear=3
enqueue(50):            [40][50]                front=3, rear=4
dequeue() -> print 40:  [50]                   front=4, rear=4
dequeue() -> print 50:  []                     front=5, rear=4
```

Output: `10 20 30 40 50`

The elements come out in the **exact same order** as they went in. This is FIFO in action!

Trace Exercise 3: Linked List Queue

Trace `enqueue(X)`, `enqueue(Y)`, `dequeue()`, `enqueue(Z)`, `dequeue()`:

```
enqueue(X):
  front --> [X|NULL] <-- rear

enqueue(Y):
  front --> [X| *]--> [Y|NULL] <-- rear

dequeue() returns X:
  front --> [Y|NULL] <-- rear       (X node freed)

enqueue(Z):
  front --> [Y| *]--> [Z|NULL] <-- rear

dequeue() returns Y:
  front --> [Z|NULL] <-- rear       (Y node freed)
```

Final Queue: Contains only `Z`. Front = Rear = Z node.

Key Takeaways

What We Learned Today

1. Queue Concept

- FIFO (First In, First Out) principle
- Two access points: front (dequeue) and rear (enqueue)
- Core operations: `enqueue()`, `dequeue()`, `peek()`, `isEmpty()`
- Error conditions: overflow (enqueue on full) and underflow (dequeue on empty)

2. Array Implementation

- Linear array with `front` and `rear` indices
- All operations O(1), but suffers from **wasted space** / false overflow
- Shifting fix makes dequeue O(n) — not acceptable
- The real fix: Circular Queue (next lecture)

What We Learned Today (cont.)

3. Linked List Implementation

- Two pointers: `front` (head) and `rear` (tail)
- Enqueue at tail, dequeue at head — both O(1)
- No size limit, no wasted space
- Edge case: must update `rear = NULL` when last element is dequeued

4. Stack vs Queue

- Stack reverses order (LIFO); Queue preserves order (FIFO)
- Stack has one access point; Queue has two
- Both can be implemented with arrays or linked lists
- Both have O(1) operations when implemented correctly

Complexity Summary

Operation	Array Queue	Linked List Queue
`enqueue()`	O(1)	O(1)
`dequeue()`	O(1)*	O(1)
`peek()`	O(1)	O(1)
`isEmpty()`	O(1)	O(1)
`isFull()`	O(1)	N/A
Space	O(n) fixed	O(n) dynamic

*Linear array dequeue is O(1) but wastes space. With shifting, it becomes O(n). The circular queue (next lecture) solves this elegantly.

Review Questions

1. What is the FIFO principle? How does it differ from LIFO?
2. In an array-based queue, what is the "wasted space" or "false overflow" problem?
3. Why do we need two pointers (`front` and `rear`) for a queue but only one (`top`) for a stack?
4. Write the `enqueue()` function for a linked list queue.
5. What happens if you forget to set `rear = NULL` when dequeuing the last element from a linked list queue?
6. If you enqueue elements 5, 10, 15, 20 and then dequeue twice, what is the front element?
7. Can a queue be implemented using two stacks? Briefly describe the idea.